# Plan de trabajo — Pipeline Transfermarkt

Oct 20, 2026 · @Naza

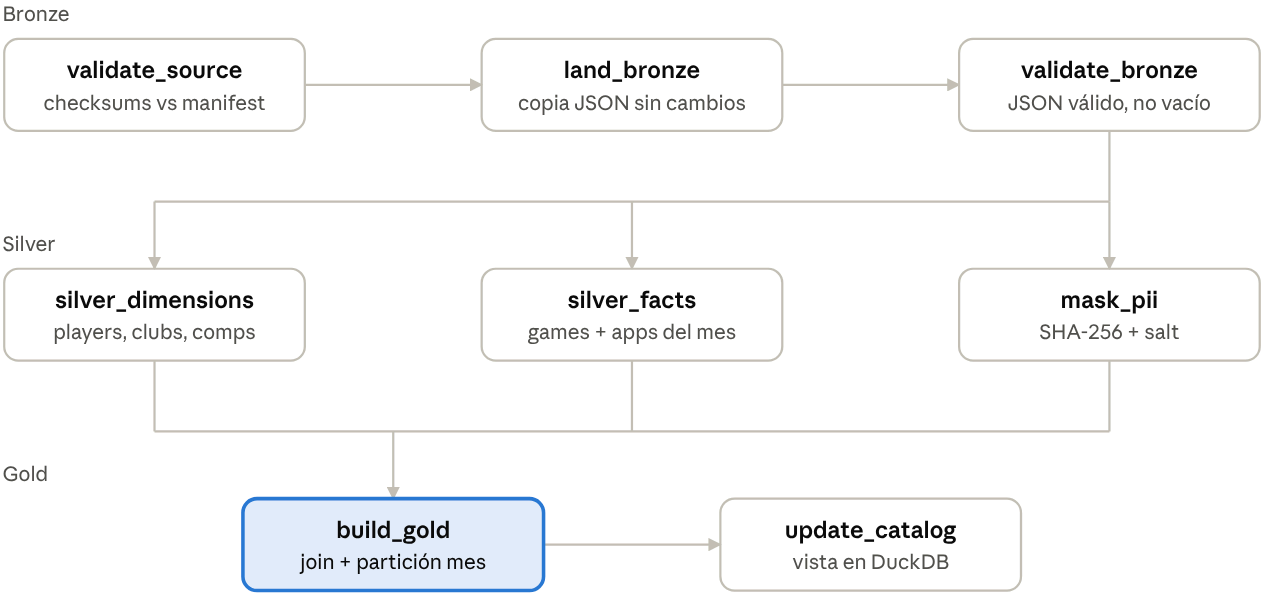
## Resumen

El proyecto construye un pipeline medallion local, orquestado con Airflow, sobre datos crudos de Transfermarkt: el JSON tal como sale del scraper, nunca los CSV ya limpios. El Gold es una tabla de apariciones de jugadores por partido, particionada por fecha del partido y consultada con DuckDB.

| Decisión | Propuesta |
|---|---|
| Fuente | JSON lines crudos del scraper open source `transfermarkt-scraper` (opciones en la sección siguiente) |
| Entidades | competitions, clubs, players, games, appearances + tabla sintética de PII |
| Recorte | Liga Profesional Argentina + 2 o 3 ligas europeas, varias temporadas; se ajusta para quedar en 50–300 MB sin comprimir |
| Business date | Fecha del partido ( `games.date` ) |
| Clave de partición Gold | `year=YYYY/month=MM` de la fecha del partido |
| Ingestion date | Fecha en que se congeló el snapshot fuente (ej. 2026-10) |
| Gold | appearances ⋈ games ⋈ players ⋈ clubs (+ competitions) |
| PII | Contactos sintéticos por `player_id` ; email hasheado con SHA-256 + salt de Airflow Variable |

La regla de oro: los datos se consiguen una vez, se congelan como "sistema fuente" y el DAG nunca vuelve a tocar Transfermarkt. Eso es lo que hace al pipeline reproducible.

**El DAG valida y copia el crudo, limpia en paralelo y arma Gold por mes**




DAG propuesto · 8 tareas, 3 capas

Las tres tareas de Silver corren en paralelo porque no dependen entre sí; separarlas permite reintentar solo la que falla.

Configuración del DAG: `schedule="@monthly"`, `start_date` en el primer mes del recorte y `catchup=False`. Los meses históricos se cargan con `airflow backfill create --dag-id <id> --from-date <inicio> --to-date <fin>`. Cada tarea toma su ventana de `data_interval_start` y `data_interval_end`. Las dependencias se declaran explícitas con `>>`:

```
validate_source >> land_bronze >> validate_bronze >> [silver_dimensions,
silver_facts, mask_pii] >> build_gold >> update_catalog
```

## Obtención de los datos crudos

Recomendación: usar la opción A como base y sumar la B, acotada, como adquisición propia. Las tres opciones dan datos reales y crudos; cambian el riesgo y el trabajo.

| Opción | Qué es | A favor | En contra |
|---|---|---|---|
| A. Snapshot crudo vía DVC | El proyecto `transfermarkt-datasets` guarda en `data/raw` la salida cruda de su scraper. Se baja con `dvc pull` desde un bucket público, sin credenciales. Licencia CC0. | Riesgo casi nulo. JSON crudo, sin limpiar. Muchas temporadas y ligas. | No lo scrapeaste vos. Datos hasta julio 2026: el autor pausó las actualizaciones. |
| B. Correr `transfermarkt-scraper` | El mismo scraper (Crawlee para Python), corrido por ustedes con `python -m tfmkt`. Saca JSON lines por stdout. | Adquisición propia. Elegís ligas y temporadas. Mismo esquema que A, así que se combinan. | Transfermarkt usa protección anti-bot (DataDome). El pipeline del autor dejó de funcionar en julio 2026 por eso. Puede tardar o fallar. |
| C. Scraper propio | `requests` + BeautifulSoup, o la librería ScraperFC, sobre las páginas que elijan. | Control total y más para contar en la defensa. | Mucho más trabajo, parsing frágil y los mismos bloqueos que B. |

## Opción A: bajar el crudo del proyecto

```
git clone https://github.com/dcaribou/transfermarkt-datasets
cd transfermarkt-datasets
pip install dvc
dvc pull data/raw/transfermarkt-scraper
```

Si DVC no acepta esa ruta como target, `dvc pull` sin argumentos trae todo y después copian solo lo necesario. Según el `config.yml` del proyecto, el crudo queda como un `.json.gz` por temporada y entidad, en formato JSON lines.

### Opción B: scrapear una liga

```
git clone https://github.com/dcaribou/transfermarkt-scraper
cd transfermarkt-scraper
poetry install && poetry shell
python -m tfmkt confederations > confederations.json
python -m tfmkt competitions -p confederations.json > competitions.json
# filtrar la Liga Profesional (verificar su ID en competitions.json)
grep '"AR1N"' competitions.json > arg.json
python -m tfmkt clubs -p arg.json --season 2024 > clubs_2024.json
python -m tfmkt players -p clubs_2024.json --season 2024 > players_2024.json
python -m tfmkt appearances -p players_2024.json --season 2024 >
appearances_2024.json
python -m tfmkt games -p arg.json --season 2024 > games_2024.json
```

Reglas para B: una liga y una temporada primero, sin paralelismo agresivo y revisando los términos del sitio. Si aparece un bloqueo, se frena ahí: no usar proxies ni servicios para evadirlo. En ese caso se sigue solo con A.

## Fase 0 — Validación y factibilidad

Objetivo: saber en el primer día si el dataset cumple la consigna y cuánto hay que recortar. Nada de Airflow todavía.

- [ ] Bajar el snapshot de la opción A.
- [ ] Medir el tamaño sin comprimir por entidad y temporada.
- [ ] Inspeccionar el crudo en un notebook: cómo vienen fechas, valores de mercado, resultados, nulos y duplicados.
- [ ] Probar la opción B con una liga y una temporada. Máximo medio día.
- [ ] Definir el recorte (ligas × temporadas) apuntando a unos 150 MB, lejos de ambos límites.
- [ ] Confirmar que hay al menos 4 entidades que joinean bien por ID.

Salida de la fase: un notebook de exploración y una decisión escrita de alcance.

## Fase 1 — Snapshot fuente y PII sintética

El crudo se congela en una carpeta fuera del repo que hace de "sistema fuente". El DAG lee de ahí y nunca de Internet.

1. Crear `~/tm_source/` con una subcarpeta por entidad y un archivo por temporada, tal como sale del scraper.
2. Escribir `manifest.json`: archivo, tamaño, SHA-256, origen (A o B) y fecha de obtención. Sirve para probar que la fuente no cambió entre corridas.
3. Generar la tabla sintética `player_contacts` con Faker y semilla fija: `player_id`, `full_name`, `email`, `phone_number`. Un registro por jugador del recorte.
   a. Semilla fija = siempre los mismos datos, así el pipeline es determinístico.
   b. Se guarda como CSV en `~/tm_source/` y va al `.gitignore`.
4. Dejar el script de generación en `/scripts`; el CSV no se commitea.

Si combinan A y B, el mismo partido o jugador puede aparecer dos veces. No se resuelve acá: es el caso de deduplicación que se trabaja en Silver.

## Fase 2 — Entorno y repositorio

Cada integrante tiene que poder correr el DAG en su máquina antes de escribir lógica de negocio.

- [ ] WSL2 + Ubuntu, Python 3.12 y Airflow 3.0.4 con el archivo de constraints, como indica la consigna.
- [ ] Instalar en el mismo entorno: `pandas`, `pyarrow`, `duckdb`, `faker`.
- [ ] `airflow variables set PII_SALT <valor>` en cada máquina. El valor se comparte por fuera del repo y no aparece en ningún archivo commiteado: ni en el DAG, ni en scripts, ni en un `.env`.
- [ ] Crear `~/airflow/data/{bronze,silver,gold}`.
- [ ] Repo con `/dags` (el DAG), `/scripts` (snapshot, PII sintética, validaciones), `/sql` (vista del catálogo y consultas DuckDB), `/docs` (salidas y capturas) y `README.md`.
- [ ] `.gitignore` con `data/`, `tm_source/`, `*.csv` de PII, `*.duckdb` y entornos virtuales.
- [ ] Ramas: `main` protegida, `develop` de integración y `feature/*` con Pull Request.
- [ ] Un DAG de prueba con dos tareas vacías que corra en todas las máquinas.

## Fase 3 — Bronze

Bronze copia los archivos del snapshot sin tocar un byte, en `bronze/{table}/ingested_year=YYYY/ingested_month=MM/`.

- La fecha de ingesta sale del `manifest.json` (cuándo se congeló el snapshot), no del reloj. Así, re-ejecutar no crea carpetas nuevas.
- Si la carpeta ya existe, se borra y se vuelve a escribir: es idempotente y barato.
- Formato: el JSON lines del scraper. Si la cátedra exige archivos sin comprimir, se descomprime el `.gz` y se documenta que el contenido no se altera.
- Tarea de validación posterior: los checksums coinciden con el manifest, no hay archivos vacíos y cada línea es JSON válido.

Punto para la defensa: la ingesta (ej. octubre 2026) es muy distinta de la fecha de negocio (partidos de 2018 en adelante). Ese contraste explica por qué Bronze y Gold se particionan por fechas diferentes.

Alternativa que pueden evaluar: un extract incremental por mes (solo los partidos del mes en cada corrida). Es más realista, pero agrega complejidad sin sumar puntos a la consigna.

## Fase 4 — Silver

Silver tipa, deduplica, resuelve nulos y enmascara PII con pandas + pyarrow, y escribe Parquet. Las dimensiones se reescriben enteras; los hechos solo para el mes de la corrida.

| Tabla | Deduplicación | Tipado esperado | Nulos y huérfanos (a validar con los datos) |
|---|---|---|---|
| players | Un registro por `player_id`: el de la temporada más reciente | Fechas de nacimiento, altura a cm, valor de mercado de texto a euros | Sin valor de mercado: queda NULL, nunca 0, para no sesgar promedios |
| clubs | Uno por `club_id`, temporada más reciente | Valores de mercado y capacidades a número | Sin estadio o DT: se mantiene, no afecta joins |
| games | Uno por `game_id`; si viene de A y de B, gana la adquisición más reciente | Fecha a datetime, resultado a goles local y visitante | Sin fecha: se descarta, no se puede particionar. Sin asistencia: NULL |

| Tabla | Deduplicación | Tipado esperado | Nulos y huérfanos (a validar con los datos) |
|---|---|---|---|
| appearances | Uno por (`player_id`, `game_id`) | Minutos, goles, tarjetas a enteros | Jugador o partido inexistente: se descarta como huérfano y se loguea el conteo |
| competitions | Uno por `competition_id` | Normalizar nombres y casing | Sin país (torneos internacionales): "Internacional" |
| player_contacts | Uno por `player_id` | — | Email nulo: se descarta el contacto, no el jugador |

Cada decisión de la tabla va al README con su justificación. Las reglas son hipótesis: se confirman en la Fase 0 mirando el crudo real.

## PII

- `email_hash = sha256(salt + email.strip().lower())`. Normalizar antes de hashear permite que dos cargas del mismo email den el mismo hash.
- `full_name` y `phone_number` se eliminan o se hashean igual; en Silver no queda ningún dato en claro.
- El salt se lee con `Variable.get("PII_SALT")` dentro de la función `@task`, nunca a nivel de módulo.

## Implementación

- Toda transformación es pandas + pyarrow dentro de funciones `@task`. DuckDB se usa solo para consultar Gold, nunca para transformar.
- Layout `silver/{table}/`. Para los hechos, un Parquet por mes dentro de esa carpeta (ej. `silver/games/2024-03.parquet`), así una re-ejecución reemplaza solo su mes.
- `player_contacts` entra a Bronze como CSV, igual que el resto del crudo, y sale de Silver ya enmascarada.

# Fase 5 — Gold y catálogo DuckDB

Gold es una tabla desnormalizada de apariciones: una fila por jugador por partido, con todo el contexto necesario para analizar sin hacer joins.

- Join: `appearances` ⋈ `games` (fecha, resultado, competición) ⋈ `players` (posición, nacionalidad, valor) ⋈ `clubs` (club y rival) ⋈ `competitions` (liga, país). Son 5 entidades de Silver; la consigna pide al menos 4.
- Partición: `gold/player_appearances/year=YYYY/month=MM/` según la fecha del partido, con el mes en dos dígitos (`month=03`).
- Escritura: pandas + pyarrow. Se borra la carpeta del mes con `shutil.rmtree` y se reescribe. Nunca append.
- Catálogo: la tarea `update_catalog` ejecuta un `.sql` de `/sql` que crea o reemplaza la vista `player_appearances` en un archivo `.duckdb` con `read_parquet('~/airflow/data/gold/player_appearances/**/*.parquet', hive_partitioning=True)`.

Por qué la fecha del partido y no la temporada: el análisis típico es por período y el mes da muchas particiones para mostrar pruning. Además, junio y julio casi no tienen partidos europeos, así que las particiones quedan desparejas. Es un buen tema para la defensa.

Por qué desnormalizar: el analista consulta siempre la misma combinación de entidades. Un solo Parquet particionado evita joins repetidos y aprovecha mejor el pruning que cinco tablas en 3FN.

# Fase 6 — Idempotencia y pruning

Dos pruebas que la consigna pide demostrar, cada una con su script en `/scripts` y su salida guardada en `/docs`.

## Idempotencia

1. Correr el DAG para un mes con muchos partidos (ej. 2024-03).
2. Contar filas en Silver (games y appearances del mes) y en la partición Gold.
3. Re-ejecutar el mismo mes desde la UI (Clear) y volver a contar.
4. Los conteos tienen que ser idénticos. Guardar ambas salidas.

Lo que lo garantiza: el DAG solo procesa `data_interval_start` a `data_interval_end`, la fuente es fija y cada capa reescribe su carpeta en lugar de agregar.

### Pruning con DuckDB

- `scripts/finops_validation.py`, como el ejemplo de la consigna, con `EXPLAIN ANALYZE` sobre todo Gold y sobre un solo mes.
- La consulta filtrada tiene que leer menos archivos y menos filas. Guardar ambas salidas una al lado de la otra.
- Antes hay que hacer backfill de varias temporadas para tener muchas particiones.

## Fase 7 — Documentación y defensa

El README tiene que responder de antemano cada pregunta de la defensa. Cada fila apunta a lo que hay que tener listo.

| Pregunta de la defensa | Dónde se responde |
|---|---|
| Idempotencia y re-ejecución | Salidas de la Fase 6 + función de escritura que borra la partición |
| Por qué estas tareas separadas | Diagrama del DAG + una línea por tarea en el README |
| Nulos y deduplicación | Tabla de decisiones de Silver con justificación |
| Clave de partición y pruning | Salida de `EXPLAIN ANALYZE` + ingesta vs. fecha de negocio |
| Salt y seguridad | Código de la tarea de PII; sin el salt, un hash no se revierte con diccionarios |
| Modelo Gold | Diagrama de joins y razón para desnormalizar |
| Migración a AWS | Sección corta (abajo) |

Migración a AWS, en una línea por componente: carpetas locales a S3; Airflow a MWAA; Variable `PII_SALT` a Secrets Manager; DuckDB a Athena con Glue Data Catalog; el snapshot fuente a un bucket de landing. En el código cambian las rutas (`s3://` vía `pyarrow.fs` o `s3fs`) y el backend de secretos; la lógica de pandas sigue igual.

## Cronograma: esta semana

Cinco días de trabajo completo, de miércoles 21 a domingo 25, con el lunes 26 como margen antes de la entrega. Si son dos o más, se reparten en paralelo las tareas de cada día.

1. Miércoles 21 — Fase 0 y Fase 2 en paralelo. Una persona baja, mide y explora el crudo y prueba la opción B hasta el mediodía. Otra instala Airflow en todas las máquinas, arma el repo y las ramas y corre el DAG de prueba. Al cierre: recorte decidido.
2. Jueves 22 — Fase 1 (snapshot congelado, manifest, PII sintética) y Fase 3 (Bronze y su validación corriendo en el DAG).
3. Viernes 23 — Fase 4: Silver de dimensiones, hechos y PII. README con las decisiones de nulos y deduplicación.
4. Sábado 24 — Fase 5: Gold, catálogo DuckDB y backfill de todas las temporadas del recorte.
5. Domingo 25 — Fase 6 (prueba de idempotencia y `EXPLAIN ANALYZE`, con salidas guardadas en `/docs`) y Fase 7 (README final).
6. Lunes 26 — Margen para imprevistos y ensayo de la defensa con preguntas cruzadas.

Hito clave: el miércoles a la noche el crudo tiene que estar bajado y el recorte decidido. Si eso se atrasa, todo el resto se corre un día.

## Chequeo contra la consigna

El plan cubre todos los requisitos técnicos. Dos dependen del crudo real y se confirman el miércoles. Seis faltaban o estaban implícitos y ya se agregaron en sus fases.

| Requisito | Cómo lo cubre el plan | Estado |
|---|---|---|
| 50–300 MB sin comprimir | Recorte medido en la Fase 0, apuntando a 150 MB | A confirmar con el crudo |
| Al menos 3 tablas joinables | 5 entidades + `player_contacts` | Cubierto |
| Columna de fecha nativa | `games.date` | Cubierto |
| Imperfecciones reales | Texto en fechas y valores, duplicados entre temporadas, huérfanos | A confirmar con el crudo |
| PII sintética antes de Bronze | `player_contacts` se genera en la Fase 1 y entra a Bronze como CSV | Cubierto |

| Requisito | Cómo lo cubre el plan | Estado |
|---|---|---|
| Estructura de carpetas | `bronze/{table}/ingested_year/ingested_month`, `silver/{table}`, `gold/{dataset}/year/month` | Cubierto |
| Bronze crudo sin modificar | Copia exacta del JSON y CSV del snapshot | Cubierto |
| Ingesta vs. fecha de negocio | Fecha del manifest vs. `games.date` | Cubierto |
| Silver solo con pandas + pyarrow en `@task` | DuckDB solo consulta, no transforma | Agregado |
| Deduplicación programática | Por clave, se queda el registro más reciente | Cubierto |
| Tipado, nulos y huérfanos documentados en README | Tabla de decisiones de la Fase 4 | Cubierto |
| SHA-256 + salt como Airflow Variable, leída dentro del `@task` | Fases 2 y 4 | Cubierto |
| El salt no aparece en ningún archivo commiteado | Checklist de la Fase 2 | Agregado |
| Silver en Parquet con pyarrow | Fase 4 | Cubierto |
| Gold con al menos 4 entidades de Silver | 5 entidades | Cubierto |
| Partición Hive `year=YYYY/month=MM` | Mes en dos dígitos ( `month=03` ) | Agregado |
| Gold expuesto a DuckDB con `hive_partitioning=True` | Tarea `update_catalog` + `.sql` en `/sql` | Cubierto |
| Al menos 5 tareas con dependencias `>>` | 8 tareas, cadena explícita | Agregado |
| Procesa solo la ventana de `data_interval_start` | Hechos de Silver y Gold filtran por mes; backfill con Airflow 3 | Agregado |
| Re-ejecución con conteos idénticos, demostrada | Prueba de la Fase 6 con salidas guardadas | Cubierto |
| Overwrite de partición, nunca append | Bronze, Silver de hechos y Gold | Cubierto |
| `EXPLAIN ANALYZE` completo vs. podado, lado a lado | `scripts/finops_validation.py` | Cubierto |
| Airflow 3.0.4 standalone en WSL2 con Python 3.12 | Fase 2 | Cubierto |

| Requisito | Cómo lo cubre el plan | Estado |
|---|---|---|
| Repo con `/dags`, `/scripts`, `/sql`, `/docs` | Contenido de cada carpeta definido | Agregado |
| `.gitignore` con datos y PII | Fase 2 | Cubierto |
| `main` protegida, `develop`, `feature/*` con PR | Fase 2 | Cubierto |
| Preguntas de la defensa, incluida migración a AWS | Fase 7 | Cubierto |

## Riesgos y planes B

| Riesgo | Qué se hace |
|---|---|
| El scraper (opción B) se bloquea o no instala | Se corta el miércoles al mediodía y se sigue solo con la opción A |
| `dvc pull` falla o el bucket deja de estar disponible | Bajar el crudo el miércoles a primera hora y guardar una copia propia |
| El recorte queda fuera de 50–300 MB | Sumar o quitar temporadas o ligas; se mide en la Fase 0 |
| El crudo viene más limpio de lo esperado | Igual hay dedup entre temporadas, huérfanos y tipado; documentarlo bien |
| El backfill de muchos meses tarda demasiado | Reducir temporadas del recorte; para la demo alcanzan dos o tres |
| Diferencias de entorno entre integrantes | Mismas versiones fijadas el miércoles y un DAG de prueba común |

## Fuentes


- transfermarkt-datasets (README, estado de actualizaciones)
- transfermarkt-datasets: guía de desarrollo (DVC, acquirers)
- transfermarkt-datasets: config.yml (formato del crudo)
- Anuncio de pausa de actualizaciones (julio 2026)
- transfermarkt-scraper (uso y crawlers)

- ScraperFC: módulo Transfermarkt